

CONTINUOUS INTEGRATION

COURSE CODE: CSE 404
COURSE TITLE: SOFTWARE ENGINEERING AND ISD
LABORATORY

Submitted by

Abrar Hameem (398)

Submitted to

Dr. Md. Musfique Anwar, Professor

Dr. Md. Humayun Kabir, Professor



Computer Science and Engineering
Jahangirnagar University

Dhaka, Bangladesh

January 7, 2025

Contents

1	Objective	1
2	Introduction	1
3	Tools for Continuous Integration and workflows	1
3.0.1	CI Tool Selection	2
3.0.2	Workflow Creation	2
4	Test-Driven Development (TDD) and Continuous Integration (CI)	3
5	Benefits of CI	4
6	Conclusion	4

1. Objective

The objective of this report is to detail my individual contributions and experiences in implementing Continuous Integration (CI) during our project. This report will highlight the tools used, the benefits achieved, and the challenges encountered, along with a reflection on how CI complemented other practices like Test-Driven Development (TDD) to improve software quality.

2. Introduction

Continuous Integration (CI) is a critical agile practice that ensures frequent and automated integration of code changes into a shared repository. By automating builds and tests, CI enables early detection of integration issues and promotes a consistent, high-quality development process. This report reflects on the tools and workflows adopted for CI in our project and how they contributed to the successful delivery of our software solution.

3. Tools for Continuous Integration and workflows

A continuous integration tool helps build and maintain clean and bug free codebase effortlessly. But choosing the right CI tool is crucial for the development process.

3.0.1 CI Tool Selection

I, along with my teammate Taufiq Islam explored three CI tools-

- **CircleCI:** Fantastic tool with support for both GitHub and Bitbucket, but has some features locked behind a paywall.
- **Jenkins:** One of the oldest and well supported CI tools to this day with universal supports, But needs hosting and server side setup.
- **GitHub Actions:** Built in easy-to-use CI tool for GitHub repositories with a marketplace for ready-made workflows, but only supports GitHub.

Since we're already storing our code in GitHub and familiar with it, we decided it was best for the team to use GitHub Actions as our CI tool.

3.0.2 Workflow Creation

A CI tool has workflows that it will execute on certain conditions. These workflows are the heart of the whole CI process. These workflow files are usually .yaml files. They describe-

- Running environment
- Workflow trigger conditions
- Processes

I, along with my teammate Taufiq Islam wrote the CI workflow for our project. our workflow included these steps-

- **Trigger:** Runs on push or pull requests to the main branch.
- **Environment:** Uses ubuntu-latest as the runner.
- **Steps:**
 - **Checkout Code:** Uses the actions/checkout@v2 action from marketplace.
 - **Set Up Node:** Installs Node.js (v18).
 - **Install Docker Compose:** Installs Docker Compose (v1.29.2).

- **Build and Start Services:** Builds and starts Docker services using docker-compose up.
- **Install Dependencies:** Installs dependencies for both client and server
- **Run Server Tests:** Executes tests for the server directory.
- **Tear Down:** Stops and removes Docker Compose services.

4. Test-Driven Development (TDD) and Continuous Integration (CI)

TDD and CI are complementary practices that work together to enhance software quality and development efficiency:

- **Alignment of Objectives:** TDD emphasizes writing all the tests first and iterating and developing the feature until all tests pass. This works nicely with CI as CI workflows reject any code that fails the tests written in TDD method.
- **Early Bug Detection:** TDD helps detect bugs early in the development process. CI extends this by not allowing any bugs during integration through its automatic testing.
- **Improved Collaboration:** TDD creates a shared understanding of feature requirements through test cases. CI reinforces collaboration by providing a stable, tested codebase for all team members to work on.

5. Benefits of CI

- **Improved Code Quality:** Automated tests and linting tools ensured adherence to quality standards, reducing bugs and inconsistencies.
- **Faster Issue Resolution:** Early detection of integration conflicts and failing tests allowed for quick fixes, reducing downtime and rework.
- **Efficient Collaboration:** CI enabled smooth collaboration among team members by providing instant feedback on code changes and ensuring a stable codebase.
- **Regular Working Releases:** The consistent integration process ensured that the project was always in a deployable state, facilitating frequent releases.

6. Conclusion

The implementation of Continuous Integration in our project significantly improved the quality and reliability of the software. By automating key processes such as testing and code integration, we minimized manual effort, reduced bugs, and ensured consistent progress throughout the project. My contributions to tool selection, pipeline design, and test integration played a pivotal role in achieving these outcomes. This experience reinforced the value of CI as a core agile practice and provided valuable insights for future projects.